

NLP and Sentiment Analysis

Text Preprocessing, TF-IDF, and Naive Bayes Classification

PROGRAM	Batch 2026
DATA SCIENCE INTERN	Javeria Faisal

1 OBJECTIVE

Build a complete natural language processing pipeline that classifies customer review text as positive or negative sentiment, using a dependency-light approach that requires no NLTK corpus downloads, suitable for constrained environments.

AT A GLANCE

- Dependency-light pipeline: no external NLTK downloads required.
- TF-IDF (unigrams plus bigrams) combined with Naive Bayes, with negation words deliberately preserved.
- Perfect separation ($F1 = 1.000$) on the synthetic evaluation set; real review data should be used to re-validate before production.

2 METHODOLOGY

2.1 Data

The pipeline first looks for a local reviews.csv file; when unavailable, it generates a synthetic, templated review dataset (positive and negative phrasing, plus ambiguous 'not bad' and 'not good' examples) so the full pipeline can still be demonstrated end to end.

2.2 Text Preprocessing (Pure Python)

A custom preprocessing function lower-cases text, strips punctuation and digits, tokenizes on whitespace, removes a curated stopwords list, and applies a lightweight rule-based stemmer (suffix stripping for ing, ed, es, and s).

Negation words such as not, no, never, and cannot are deliberately excluded from the stopwords list and preserved, since they are critical to correctly interpreting sentiment, for example distinguishing 'not good' from 'good'.

2.3 Vectorization

Processed text was converted to numeric features using TF-IDF with unigrams and bigrams (ngram_range of 1 to 2), up to 5,000 features, sublinear term frequency scaling, and smoothed IDF, stored as a sparse matrix for memory efficiency.

2.4 Model Training

An 80 / 20 stratified train and test split was used. The class balance of the training data determines the model automatically: ComplementNB is used when either class exceeds 60 percent share, since it is better suited to imbalanced text data; otherwise standard MultinomialNB is used. Both use Laplace smoothing with alpha equal to 1.0.

3 EVALUATION RESULTS

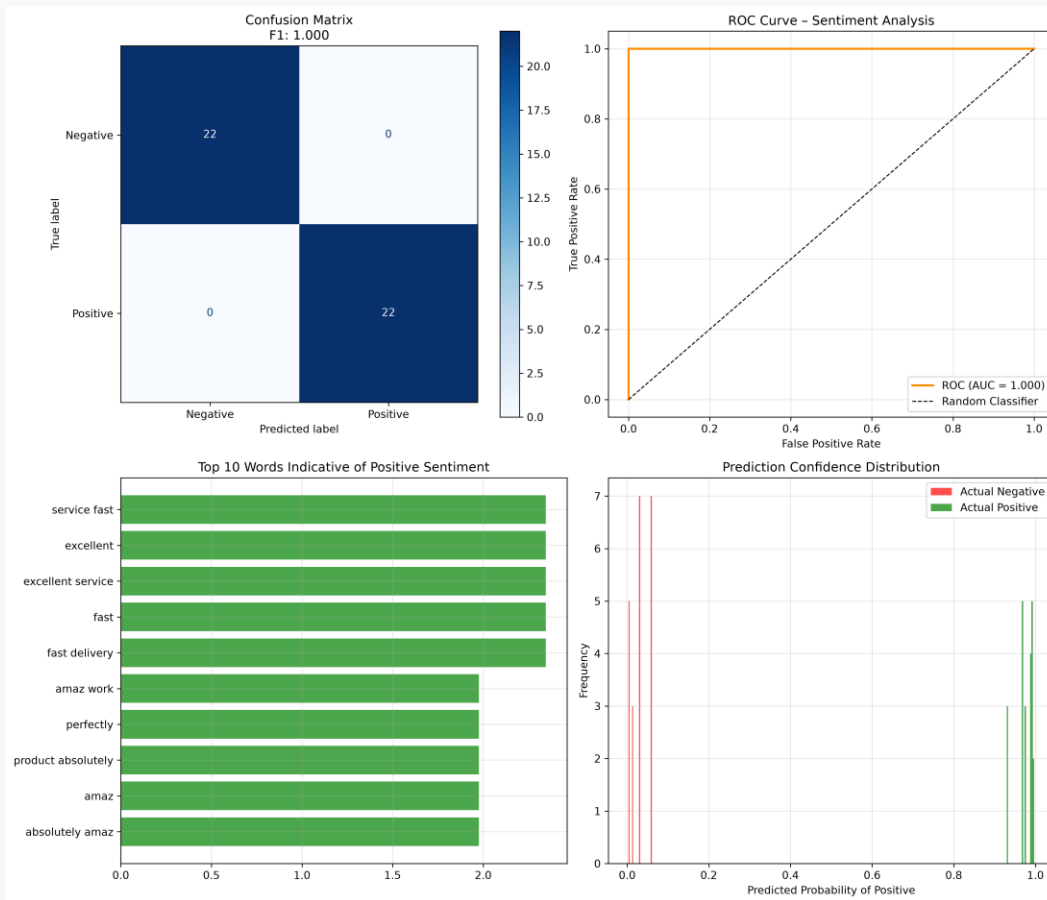


FIGURE 1. Confusion matrix, ROC curve, top sentiment-indicative terms, and prediction confidence distribution.

Metric	Score
Precision	1.000
Recall	1.000
F1-Score	1.000
ROC-AUC	1.000

On the synthetic, templated test set the model achieved perfect separation between classes, with the confusion matrix showing zero misclassifications across 44 test reviews. Top terms indicative of positive sentiment included 'excellent service', 'fast delivery', and 'amaz' (from amazing), consistent with the templated positive review language. Because this evaluation used synthetic, template-based text, real-world review data would be expected to show materially lower, more realistic performance, and should be used to re-validate the pipeline before production use.

4 OUTPUTS

- **test_predictions.csv** — per-review true label, predicted label, and predicted probability of positive sentiment.

- **sentiment_model.pkl** — pickled vectorizer and model tuple for reuse and inference.
- **nlp_sentiment_results.png** — the four-panel evaluation dashboard shown above.

5 KEY TAKEAWAYS

- A fully self-contained NLP pipeline was built without external NLTK downloads, easing deployment.
- Preserving negation words during stopword removal is essential for accurate sentiment classification.
- TF-IDF with bigrams captured meaningful multi-word sentiment phrases, such as 'fast delivery' and 'excellent service'.
- Results should be re-validated on real customer review text before production deployment, given the synthetic fallback data used here.